

Systemanalyse

Lernskript zur Vorbereitung auf die mündliche Prüfung

Stand: 12.6.2026

Inhaltsverzeichnis

Inhaltsverzeichnis	2
1. Softwarekrisen	3
Typische Ursachen für Projektscheitern	3
2. Die 3 Regeln der Softwareentwicklung	3
Regel 1: Klartext	3
Regel 2: Gründlichkeit in der Vertragsgestaltung	4
Regel 3: Umgang mit wandelnden Anforderungen	4
3. Phasen der Softwareentwicklung	4
4. Tools beim Programmieren	5
4.1 Kanbanboard	5
4.2 Storyboard	6
4.3 User-Story Board	6
4.4 Magisches Dreieck	7
4.5 Pairing-Tabelle	7
5. Prozess-Modelle (Vorgehensmodelle)	8
5.1 Wasserfallmodell (Phasenmodell)	8
5.2 Iteriertes Phasenmodell	9
5.3 Spiralen-Modell	9
5.4 V-Modell	10
5.5 V-Modell XT (2005)	11
5.6 Burndown-Chart	12
6. SCRUM	12
6.1 SCRUM-Ablauf	12
6.2 SCRUM-Rollen	13
6.3 SCRUM-Manifest (2001)	13
6.4 SCRUM-Poker	13
6.5 Team Velocity & Story Points	13
7. Wichtige Begriffe	14
Qualitätsverbesserung: 4 Säulen	14
8. Kurz-Wiederholung für die mündliche Prüfung	14

1. Softwarekrisen

Die Softwarekrisen beschreiben historische und aktuelle Probleme bei der Softwareentwicklung, die aus der wachsenden Komplexität und gesellschaftlichen Bedeutung von Software entstehen.

Krise	Ursache / Folgen / Probleme
Software-Krise I (1960er)	Ursache: Schnellere Hardware macht Software wichtiger. Probleme: niedrige Qualität, intransparente Systeme, fehlendes Fachpersonal, Kosten/Termine überschritten. Softwarekosten übersteigen Hardwarekosten.
Software-Krise II	Ursache: Entwickler- und Anwenderumgebung unterscheiden sich. Bsp.: Programm funktioniert beim Entwickler, aber nicht beim Kunden (fehlende DLL, andere Infrastruktur).
Software-Krise III	Ursache: Mehr 'smarte' Geräte in einem Netz. Folgen: Parallele Prozesse, Auslastungsprobleme.
Software-Krise IV (Zukunft)	Ursache: KI-generierter Code. Folge: Ungeprüfter, unkontrollierter Code gelangt in Produktion.

	Bekannte Beispiele gescheiterter Projekte: Mariner 1 (1962), Toll Collect (+3,5 Mrd. EUR Ausfall), FBI Virtual Case File (170 Mio. USD), Londoner Börse Taurus (+8 Mrd. GBP, Budgetüberschreitung 13.200%).
---	---

Typische Ursachen für Projektscheitern

- Unzureichend spezifizierte oder häufig geänderte Anforderungen
- Mangel an Ressourcen oder kompetenten Mitarbeitern
- Fehlende Benutzereinbeziehung und Managementunterstützung
- Falsche Zeitschätzungen und überzogene Erwartungen
- Veraltete Projekte (inzwischen bessere Lösungen vorhanden)

	Klausur: 'Nennen Sie ein Beispiel, was nicht funktioniert hat, und was Sie dagegen gemacht haben.' Ausserdem: 'An welchen Phasen der Softwareentwicklung ist ein Projekt gescheitert – argumentieren Sie!'
---	---

2. Die 3 Regeln der Softwareentwicklung

Die drei zentralen Regeln definieren, wie Kommunikation, Verträge und Anforderungsmanagement professionell gehandhabt werden sollen.

Regel 1: Klartext

Alle Beteiligten müssen dasselbe meinen, wenn sie dieselben Worte verwenden. Missverständnisse entstehen durch unterschiedliche Erfahrungswelten.

Ansatz	Massnahme
--------	-----------

Wasserfallmodell	Lastenheft (Kunde schreibt auf, was er will) + Pflichtenheft (Entwickler beschreibt, was er verstanden hat und umsetzt). Muss von mindestens 3 Personen gelesen, verstanden und unterschrieben werden.
Agile Entwicklung	Backlog im User-Story-Format + Refinement (Backlog-Grooming). Iterativer Austausch ersetzt langes Vordokumentieren.

! Beispiel für Kommunikationsfehler: Was ist ein 'leerer String'? Je nach Sprache: "" (leer), " " (Leerzeichen), "\0" (FORTRAN vs C++). Gleiche Begriffe können völlig unterschiedliche Bedeutungen haben!

Regel 2: Gründlichkeit in der Vertragsgestaltung

Ansatz	Massnahme
Wasserfallmodell	Ca. 1/2 Jahr für rechtssichere Formulierung einplanen. Glossar anlegen, gleiche Begriffe abklären (LH, PH). Lastenheft und Pflichtenheft von 3 Personen prüfen und unterschreiben lassen.
Agile Entwicklung	Backlog + Refinement ersetzen den schwerfälligen Vertragsprozess. Aufgaben werden laufend präzisiert.

Regel 3: Umgang mit wandelnden Anforderungen

Änderungen sind unvermeidlich. Der Prozess muss klar regeln, wie damit umgegangen wird.

Ansatz	Massnahme
Lineares Modell	Change-Request-Prozess: (1) Anforderung formulieren, (2) Auswirkungen prüfen, (3) Bestätigung oder Ablehnung, (4) Implementierung.
Agile Entwicklung	Neues Item ins Backlog aufnehmen, in nächster Iteration priorisieren. Keine Zusatzleistungen per Mail oder mündlich ohne Backlog-Eintrag.

3. Phasen der Softwareentwicklung

Alle Vorgehensmodelle enthalten im Kern dieselben Phasen. Im Wasserfall werden sie sequenziell durchlaufen, im agilen Kontext iterativ – wie ein Tortenstück mit allen Schichten in jeder Iteration.

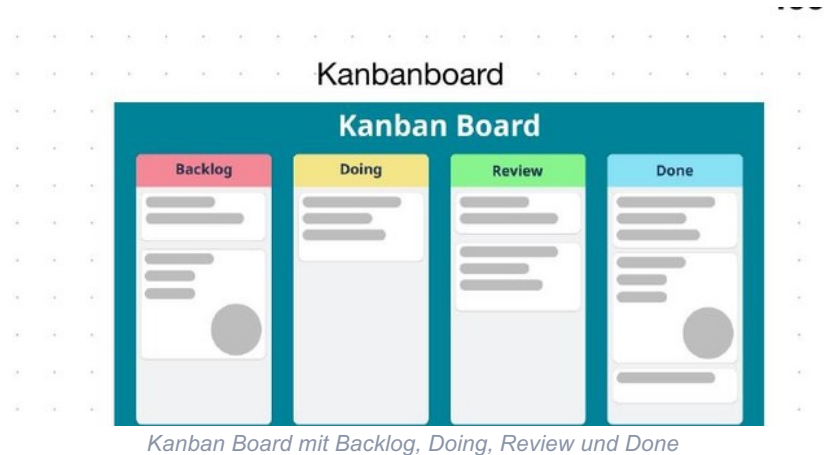
Phase	Name	Inhalt
1	Anforderungs-/Planungsphase	Kunden einbeziehen, Lastenheft erstellen, Zeitplanung.
2	Analysephase / Definitionen	Anforderungen detaillieren, Glossar, Systemgrenzen.
3	Konzeption / Entwurf / Design	Grob- und Feinentwurf, Architektur, Pflichtenheft.
4	Realisierungsphase / Entwicklung	Implementierung + Unit-Tests.
5	Integration + Test	Integrationstests, Schnittstellentests, Systemtests.
6	Auslieferung + Tests	Abnahmetests, Rollout.
7	Wartung	Fehlerbehebung, Updates, Weiterentwicklung.



Im agilen Kontext: Stell dir eine Torte vor. Die Phasen sind die Schichten – jede Iteration (Stück) enthält alle Schichten von Anforderung bis Test.

4. Tools beim Programmieren

4.1 Kanbanboard

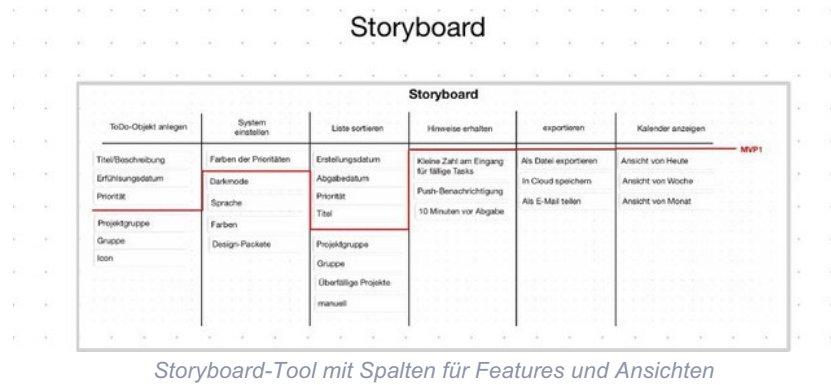


Begriff	Bedeutung
Doing / WIP	Work in Process: Aufgaben, die gerade aktiv bearbeitet werden.
WIP-Limit	Künstliches Limit für gleichzeitig bearbeitete Aufgaben. Fördert Wissenstransfer und Fokus.
Priorisierung	Reihenfolge im Backlog bestimmt die Priorität. Wichtigste Aufgabe liegt oben.
DoD	Definition of Done: Kriterien, die erfüllt sein müssen, damit eine Aufgabe als 'Done' gilt.
DoR	Definition of Ready: Kriterien, damit eine Aufgabe von Backlog nach Doing wandern darf.
Tabula Rasa	Board wird geleert. Im agilen Zyklus: alles was im Release ist, kommt zurück ins Backlog.
Test-Review Spalte	Immer als eigene Spalte behandeln (nicht in Done zusammenfassen).



Das Kanbanboard dient der Visualisierung! Probleme und Fortschritt werden für alle sichtbar gemacht. 'Früh wie möglich scheitern' ist das Ziel.

4.2 Storyboard



Das Storyboard zeigt ausschließlich die Nutzersicht – nicht die Sicht des Entwicklers. Es visualisiert, welche Features existieren und wie die Nutzerreise aussieht.

4.3 User-Story Board

User Story - Schablone

USER STORY

Als [Rolle]

möchte ich [Funktion/Ziel],

damit [Nutzen/Mehrwert].

Akzeptanzkriterien:

- ✓ Das System muss ... ermöglichen.
- ✓ Der Nutzer soll ... sehen können.
- ✓ Bei Fehler erscheint ... Meldung.

Story Points: ___ SP **Priorität:** ☐ Hoch ☐ Mittel ☐ Niedrig

User-Story-Format: Als [Nutzer] möchte ich [Funktion], damit [Nutzen]

User Stories beschreiben Probleme aus Nutzersicht nach dem Format:

"Als [Nutzerrolle] möchte ich [Funktion], damit [Nutzen]."

Beispiel aus der Vorlesung:

Als Hausmeister möchte ich ein Auto, um einen Kasten Bier in die WG zu transportieren.

- Sprint 1: 4 Räder -> Skateboard (nicht optimal, aber ausprobierbar)
- Sprint 2: Motor -> Roller
- Sprint 3: Türen -> Fahrrad
- Sprint 4: Auto final

★ Warum User-Story-Format? -> Soll iterativ sein. Das Problem des Nutzers beschreiben. Immer Fortschritt durch Feedback. Wird abgefragt: WARUM wird das Format verwendet (nicht das Format selbst).

4.4 Magisches Dreieck



Es ist nie möglich, alle drei Faktoren gleichzeitig zu maximieren. Verbesserung eines Faktors beeinträchtigt immer mindestens einen der anderen.

Faktor	Beschreibung
Leistung / Ergebnis	Qualität und Umfang des Produkts.
Aufwand (Kosten)	Ressourcen, Budget, Personal.
Zeit (Termin)	Zeitplan, Deadline, Dauer des Projekts.

4.5 Pairing-Tabelle

Pairing-Tabelle (Pair Programming)

Alice	Bob	Carol	Dave	Eve
Alice	Sprint 1	Sprint 2	Sprint 3	
Sprint 1	—		Sprint 2	
Sprint 2		—		Sprint 3
Sprint 3	Sprint 2		—	Sprint 1
Eve		Sprint 3	Sprint 1	—

Sprint 1
 Sprint 2
 Sprint 3

Pairing-Tabelle: Hut = Verantwortliche Person, Kreis = Mitarbeitende

Die Pairing-Tabelle zeigt, wer bei welchem Thema die Verantwortung (Hut = Leitung) trägt und wer mitarbeitet (Kreis). Spalten zeigen Mitarbeiter, Zeilen Themen.

5. Prozess-Modelle (Vorgehensmodelle)

Vorgehensmodelle sind standardisierte Leitfäden für die Softwareentwicklung. Sie geben an, in welcher Reihenfolge und wie die Phasen durchlaufen werden. Ziel aller Modelle: Zeit, Kosten und Leistung optimieren.

5.1 Wasserfallmodell (Phasenmodell)

Wasserfallmodell (Phasenmodell)



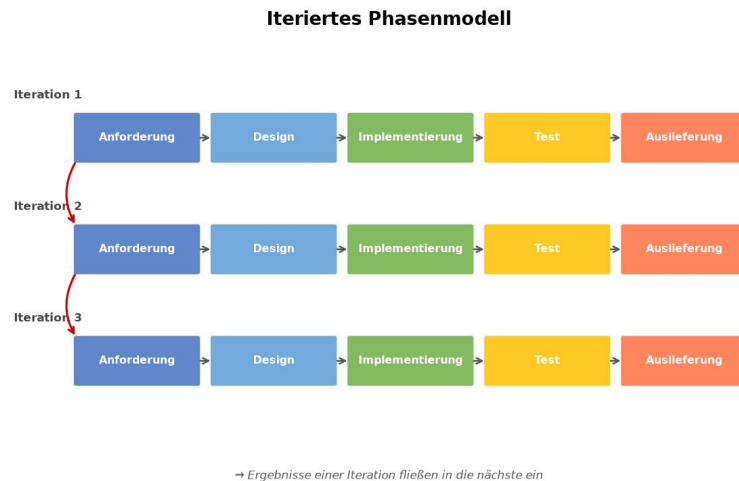
→ Sequentiell, kein Rückschritt vorgesehen

Wasserfallmodell: Lineare sequenzielle Phasen

	Wasserfallmodell	
Vorteile	Einzelabschnitte klar getrennt	
	Weniger komplex, guter Einstieg	
Nachteile	Fehlerfortpflanzung / keine Rückwirkung	
	Kein frühzeitiges Scheitern	
	Kein Wissenstransfer zwischen Phasen	

Im Wasserfallmodell entstehen: Lastenheft (vom Kunden) und Pflichtenheft (vom Entwickler).

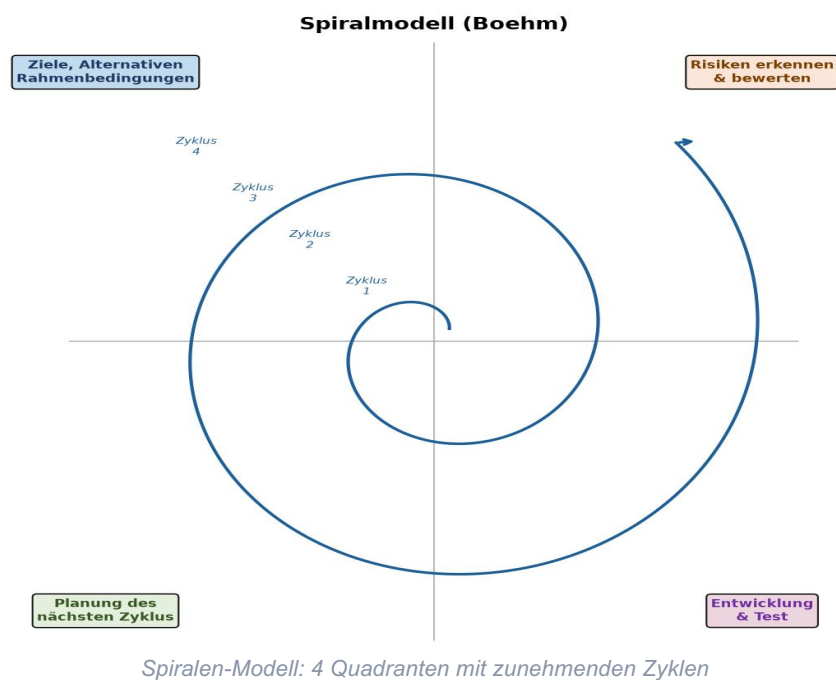
5.2 Iteriertes Phasenmodell



Iteriertes Modell: Rückwirkung zwischen den Phasen möglich

	Iteriertes Modell
Vorteile	Einzelabschnitte + Rückwirkung auf vorherige Phase möglich.
Nachteile	Dauernder Loop zurück kann ineffizient werden.

5.3 Spiralen-Modell

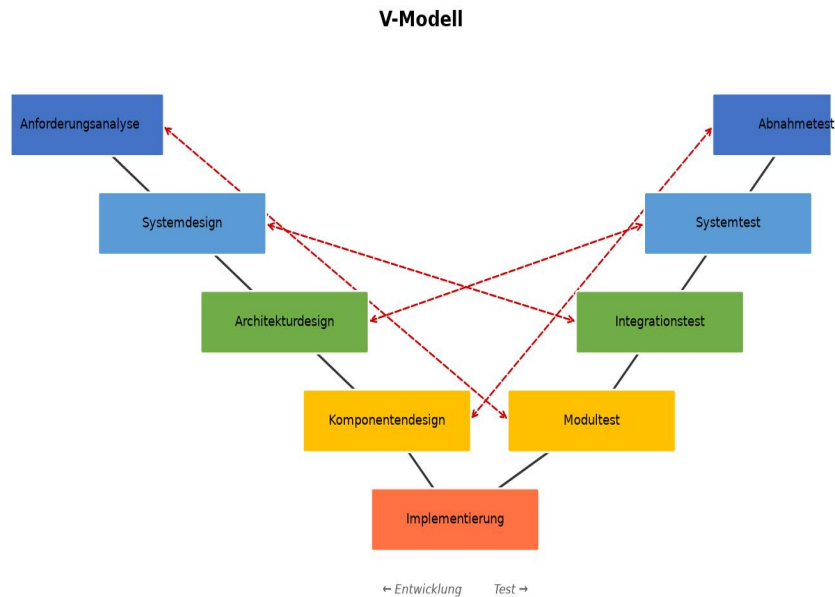


Das Spiralen-Modell ist ein iteratives Modell mit 4 Quadranten pro Zyklus:

1. Ziele, Anforderungen & Rahmenbedingungen festlegen
2. Evaluierung der Alternativen, Risikoanalyse (USP / Alleinstellungsmerkmal)
3. Realisierung & Überprüfung (Entwicklung, Test)
4. Plan des nächsten Zyklus

	Spiralen-Modell
Vorteile	Kontinuierliche Risikoanalyse. Frühes Feedback ('Fail early').
Nachteile	Höhere Komplexität und Verwaltungsaufwand.

5.4 V-Modell



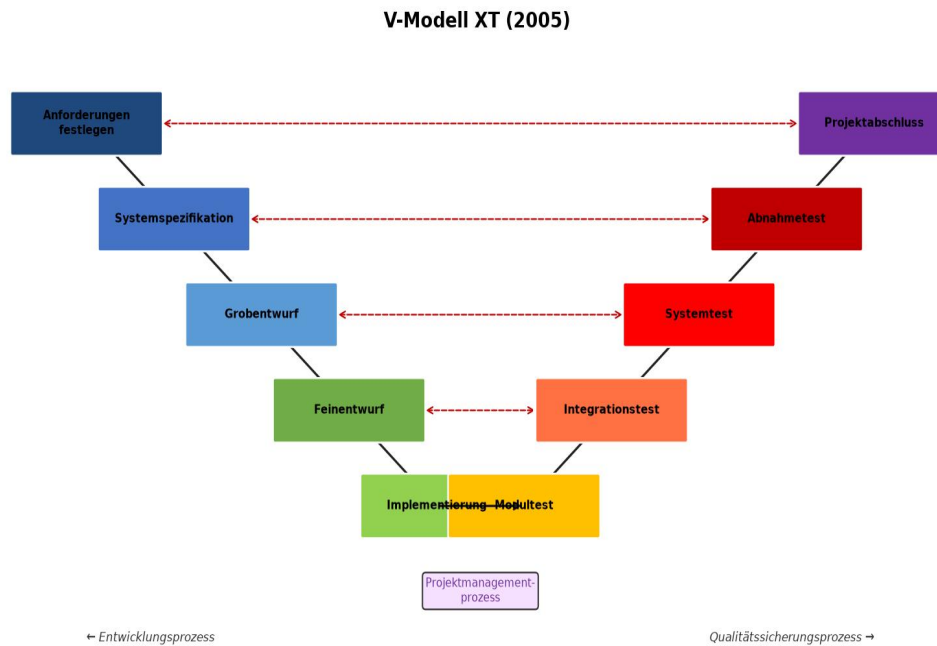
V-Modell: Jede Entwicklungsphase hat eine entsprechende Testphase

Das V-Modell verknüpft jede Entwicklungsphase direkt mit einer Testphase:

Entwicklungsseite (links)		Testseite (rechts)
Analyse	<-->	Akzeptanz-Tests
High-Level-Design	<-->	System-Tests
Low-Level-Design	<-->	Integrations-Tests
Implementierung	<-->	Unit-Tests

	V-Modell
Vorteile	Jede Phase hat eigene Testphase -> hohe Qualitätssicherung. Frühe Planung der Tests.
Nachteile	Mittlere Komplexität. Kaum Kundenkontakt während der Entwicklung (Sisyphusarbeit).

5.5 V-Modell XT (2005)

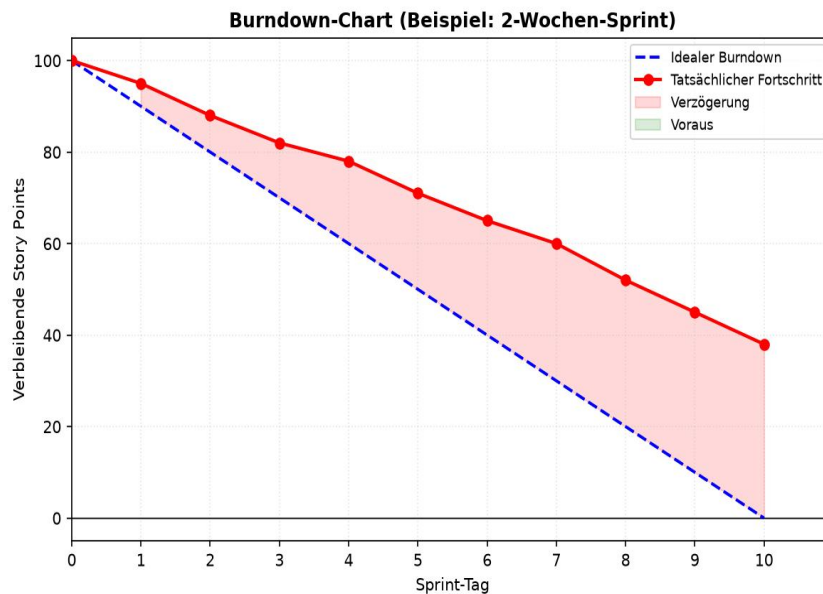


V-Modell XT: Erweiterte Version für öffentliche IT-Projekte

Das V-Modell XT entstand aus dem 'V-Modell 93' (für öffentliche IT-Projekte in Deutschland) und wurde 2005 mit mehr agilen Elementen erweitert.

Merkmal	Beschreibung
Herkunft	Aus dem Militärbereich. Richtschnur für IT-Projekte im BRD.
Ziel	Kosten transparent und limitierbar machen. Auftraggeber aktiv einbinden.
Extreme Tailoring	Das Modell soll an spezifische Projekte anpassbar sein.
Stärkere Modularisierung	Einzelne Teile können unabhängig eingesetzt werden.
Rollen	Trennung in Auftraggeber (AG) und Auftragnehmer (AN). 4 Projekttypen: AG-Projekt, AN-Projekt, AG/AN-Projekt, Organisationsspezifisches Vorgehensprojekt.
3 Säulen	Meta-Modell (Rollen, Produkte, Entitäten) + Freie Inhalte + Werkzeuge (V-Modell-XT-Editor, Projektassistenten).

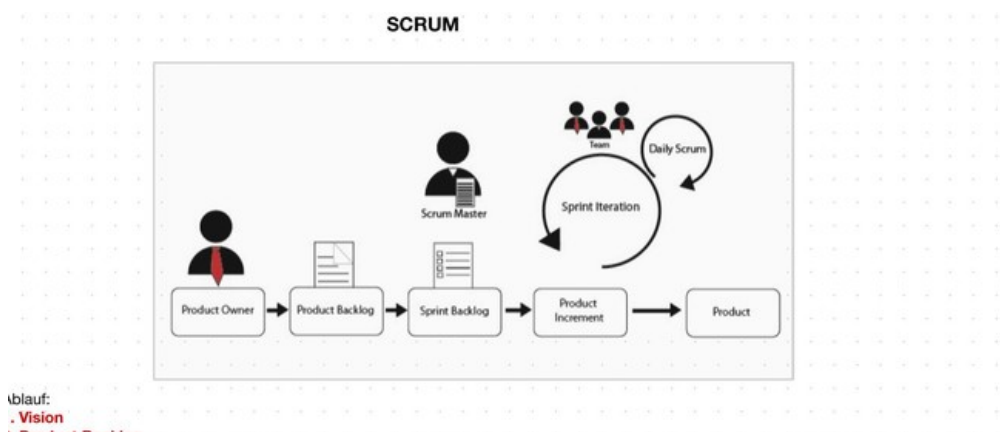
5.6 Burndown-Chart



Burndown-Chart: Verbleibender Aufwand (h) über Zeit

Das Burndown-Chart zeigt den Prozessfortschritt im Verhältnis zum übrigen Arbeitspotenzial (in Stunden oder Story Points). Im Idealfall ist der Verlauf linear. Die Realität weicht davon ab.

6. SCRUM



SCRUM-Prozess: Product Owner -> Backlog -> Sprint -> Daily Scrum -> Inkrement

6.1 SCRUM-Ablauf

Schritt	Event	Beteiligte	Verantwortung
1	Vision	Alle	Product Owner
2	Product Backlog erstellen	Product Owner	Product Owner
3	Planning Event (WAS & WIE)	PO + Scrum Master + Team	Product Owner
4	Daily Scrum (15 min täglich)	Team	Scrum Master
5	Retrospektive / Review	Team + PO + SM + Kunde	Scrum Master
6	Tabula Rasa (Board leeren)	Team	-
7	Refinement / Grooming	Team + PO	Product Owner

Retrospektive = Prozessverbesserungen erfassen (keine inhaltlichen Bewertungen!). Tabula Rasa = alles was im Release ist, kommt zurück ins Backlog.

6.2 SCRUM-Rollen

Rolle	Beschreibung
Product Owner	Schnittstelle zwischen Entwicklung und ROI (Return on Investment). Priorisiert das Backlog.
Entwickler-Team	T-Shaped Mitarbeiter: tiefes Expertenwissen in einem Bereich + breites Basiswissen.
Scrum-Master	Kummerer, Prozessbeobachter, Diener und Führer zugleich. Beseitigt Hindernisse.
Management	Stellt sicher, dass der Prozess im Unternehmen akzeptiert und unterstützt wird.
Kunde	Aktiv in den Prozess eingebunden. Feedback kommt laufend, nicht erst am Ende.

6.3 SCRUM-Manifest (2001)

5. Individuen über Tools & Prozesse
6. Produkt über Dokumentation
7. Zusammenarbeit über Vertragshandlungen

Das SCRUM-Manifest ist eine öffentliche Erklärung von Zielen und Absichten. Es dient als Entscheidungshilfe und wurde 2001 formuliert.

6.4 SCRUM-Poker

Beim SCRUM-Poker werden Schätzungen (z. B. Zeitaufwand) mit verdeckten Karten getroffen, um Gruppendenken zu vermeiden. Jedes Teammitglied hat eine gleichwertige Stimme.

Kartenwerte	0 – 0,5 – 1 – 2 – 3 – 5 – 8 – 13 – 20 – 40 – 100
Logik	Groessere Zahlen = ungenauere Schätzung ist akzeptabel (Differenzierung wird unwichtiger je größer die Zahl).

6.5 Team Velocity & Story Points

Die Team Velocity beschreibt, wie viele Story Points ein Team in einem Sprint (z.B. 2 Wochen) schafft. Story Points werden aus Erfahrung geschätzt und dem Kunden als Planungsgrundlage kommuniziert.

- WIP-Limit wird bewusst kleiner gesetzt als Teamgroesse -> fördert Wissenstransfer
- Beispiel: Team mit 17 MA, WIP auf 8 gesetzt -> Paare/Gruppen können Wissen teilen
- Dokumentation, Code-Review, Team-Meetings: Instrumente um Wissenstransfer zu sichern

7. Wichtige Begriffe

Begriff	Bedeutung
Extreme Programming (XP)	Programmiertechnik: Review (Paarprogrammierung), Testen (testgetriebene Entwicklung), Deployen (kurze Release-Zyklen), Redesign (sofortiges Refactoring) werden ins Extreme geführt. 2 Entwickler, 4-Augen-Prinzip.
MVP (Minimal Viable Product)	Softwareversion mit minimalen Features. Mehrere Level: MVP1, MVP2, MVP3 etc. Wichtig: Beispiel für eigenes Unternehmen überlegen! (Klausur)
Pretotyp	'Fake it before you make it' -> Ziel: Fail early. Billigste Version des Prototyps (z.B. 3D-Skizze), um möglichst früh zu scheitern.
Prototyp	'Quick and dirty' -> erste praktische Version mit eingeschränkten Features. Anforderungen werden geklärt, ausgewählte Eigenschaften werden demonstriert.
3-M-Methode	Mensch (Kompetenz), Maschine (Geräte, Tools), Methode (Prozesse). An einem der drei Faktoren könnte ein Projekt scheitern.
Singleton Pattern	Nur eine Instanz in der Klasse. Beispiel für ein Entwurfsmuster.
Change Request	Formaler Prozess für Änderungsanforderungen: Anforderung -> Auswirkungen prüfen -> Bestätigung/Ablehnung -> Implementierung.

Qualitätsverbesserung: 4 Säulen

Säule	Inhalt
Kompetenz der Entwickler	Fachwissen der Mitarbeiter gezielt aufbauen und erhalten.
Prozess	Systemanalyse, 4-Augen-Prinzip, Messungen, Kundenfeedback laufend einholen.
Tooling / Wiederverwendung	Bewährte Entwurfsmuster, vorhandene Lösungen nutzen, nicht neu erfinden.
Automatisierung	CI/CD, DevOps, Git-Repository. Manuelle Fehlerquellen eliminieren.

8. Kurz-Wiederholung für die mündliche Prüfung

Sprich die Antworten laut aus und prüfe, ob du sie ohne Ablesen erklären kannst.

Frage	Kern der Antwort
Was bedeutet agil?	Auf Änderungen reagieren können. Iteratives Modell, feste Struktur, klare Regeln, Backlog, User Story, Refinement.
Was ist ein Kanbanboard?	Visualisierungswerkzeug für Aufgaben (Backlog -> Doing -> Review -> Done). WIP-Limit, DoD, DoR, Tabula Rasa.
Was ist WIP-Limit?	Künstliche Begrenzung gleichzeitig bearbeiteter Aufgaben. Fördert Fokus und Wissenstransfer.
Was ist DoD / DoR?	DoD = Kriterien um Done zu erreichen. DoR = Kriterien um mit einer Aufgabe zu starten.

Wasserfall vs. agil?	Wasserfall: linear, Lastenheft/Pflichtenheft, kein Rücksprung. Agil: iterativ, Backlog, User Stories, laufendes Kundenfeedback.
Was ist MVP?	Minimal Viable Product: minimales lauffähiges Produkt. Immer mit eigenem Unternehmensbeispiel erklären!
Pretotyp vs. Prototyp?	Pretotyp: 'Fake it before you make it', billigste Version, Fail early. Prototyp: quick & dirty, erste praktische Version.
Was sind die 3 Regeln?	1. Klartext (gleiche Sprache), 2. Gründlichkeit im Vertrag (LH, PH), 3. Umgang mit wandelnden Anforderungen (Change Request / Backlog).
Was ist SCRUM-Manifest?	2001: Individuen über Tools, Produkt über Doku, Zusammenarbeit über Vertragshandlungen.
Spiralen-Modell: 4 Quadranten?	1. Ziele/Anforderungen, 2. Risikoanalyse/Evaluierung, 3. Realisierung/Test, 4. Plan des nächsten Zyklus.
V-Modell-XT Ziel?	Kosten transparent machen, Auftraggeber einbinden, extreme Tailoring (Anpassbarkeit), stärkere Modularisierung.

★ Typische Prüfungsfragen: 'Was bedeutet agil?', 'MVP-Beispiel beschreiben', 'Vor-/Nachteile Wasserfallmodell', 'Woran scheitern IT-Projekte?', 'Welche Softwareentwicklungsherangehensweise nutzt dein Praxispartner?'

Viel Erfolg bei der Prüfung! 🎓